

Вычисление наибольшего общего делителя

Филибер Никола

13 окт 2026, Москва, Россия

Российский Университет Дружбы Народов

Цель работы

Изучение и практическое применение методов программной реализации Алгоритмов вычисления наибольшего общего делителя.

Задание

1. Реализовать алгоритм Евклида.
2. Реализовать бинарный алгоритм Евклида.
3. Реализовать расширенный алгоритм Евклида.
4. Реализовать расширенный бинарный алгоритм Евклида.

Выполнение лабораторной работы

алгоритм Евклида

```
# Euclid's Algorithm (remainder / division method)
def gcd_euclid(a: int, b: int) -> int:
    a, b = abs(a), abs(b)
    if a == 0:
        return b
    if b == 0:
        return a

    while b != 0:
        a, b = b, a % b
    return a

gcd = gcd_euclid(45, 10)
print("GCD", gcd)

GCD 5
```

Рис. 1: Алгоритм Евклида

Бинарный алгоритм Евклида

```
#Binary GCD (Stein's algorithm)
def gcd_binary(a: int, b: int) -> int:

    a, b = abs(a), abs(b)
    if a == 0:
        return b
    if b == 0:
        return a

    # count common factors of 2
    shift = 0
    while ((a | b) & 1) == 0: # both even
        a >>= 1
        b >>= 1
        shift += 1

    # make a odd
    while (a & 1) == 0:
        a >>= 1

    while b != 0:
        # make b odd
        while (b & 1) == 0:
            b >>= 1

        # now a and b are odd
        if a > b:
            a, b = b, a
        b = b - a # b becomes even (difference of odds)

    return a << shift

gcd = gcd_binary(45, 10)
print(gcd)
```

5

Рис. 2: Бинарный алгоритм Евклида

Расширенный алгоритм Евклида.

```
#Extended Euclid (returns gcd, x, y)
def extended_gcd_euclid(a: int, b: int):
    if b == 0:
        g = abs(a)
        # make x carry the sign of a so identity holds
        x = 1 if a > 0 else -1 if a < 0 else 0
        return g, x, 0

    old_r, r = a, b
    old_x, x = 1, 0
    old_y, y = 0, 1

    while r != 0:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_x, x = x, old_x - q * x
        old_y, y = y, old_y - q * y

    g = abs(old_r)
    # Normalize sign so g is positive
    if old_r < 0:
        old_x, old_y = -old_x, -old_y

    return g, old_x, old_y

gcd = extended_gcd_euclid(45, 10)
print(gcd)

(5, 1, -4)
```

Рис. 3: Расширенный алгоритм Евклида

Расширенный бинарный алгоритм Евклида.

```
def extended_gcd_binary(a: int, b: int):

    if a == 0:
        return abs(b), 0, 1 if b > 0 else -1 if b < 0 else 0
    if b == 0:
        return abs(a), 1 if a > 0 else -1 if a < 0 else 0, 0

    # Work with positive values for the binary part, but keep originals for identity.
    a0, b0 = a, b
    a, b = abs(a), abs(b)

    # Remove common factors of 2
    shift = 0
    while ((a | b) & 1) == 0:
        a >>= 1
        b >>= 1
        shift += 1

    # u = a, v = b
    u, v = a, b
    A, B = 1, 0
    C, D = 0, 1

    while u != 0:
        # while u even
        while (u & 1) == 0:
            u >>= 1
            if (A & 1) == 0 and (B & 1) == 0:
                A >>= 1
                B >>= 1
            else:
                A = (A + b) >> 1
                B = (B - a) >> 1

        # while v even
        while (v & 1) == 0:
            v >>= 1
            if (C & 1) == 0 and (D & 1) == 0:
                C >>= 1
                D >>= 1
            else:
```

Расширенный бинарный алгоритм Евклида.

```

    A = (A + b) >> 1
    B = (B - a) >> 1

# while v even
while (v & 1) == 0:
    v >>= 1
    if (C & 1) == 0 and (D & 1) == 0:
        C >>= 1
        D >>= 1
    else:
        C = (C + b) >> 1
        D = (D - a) >> 1

# subtract
if u >= v:
    u = u - v
    A = A - C
    B = B - D
else:
    v = v - u
    C = C - A
    D = D - B

g = v << shift # restore factors of 2

# Now we have: a*C + b*D = gcd(a,b) where a,b are abs() reduced ones.
# Fix signs according to original a0, b0:
x, y = C, D
if a0 < 0:
    x = -x
if b0 < 0:
    y = -y

return g, x, y
```

```
: gcd = extended_gcd_binary(45, 10)
: print(gcd)
: (5, 5, -22)
```

Выводы

Изученил и разработал методы программной реализации Алгоритмов вычисления наибольшего общего делителя.

Список литературы

https://en.wikipedia.org/wiki/Euclidean_algorithm